



Guía de Estudio: Interfaces Gráficas en Java

Desarrollo con JavaFX, IntelliJ IDEA, FXML y Arquitectura MVC (Indicadores IL 2.1 – IL 2.8)

Objetivo de Aprendizaje: Entender el propósito y la relación entre cada componente de software para dominar los indicadores IL 2.1–2.8.

TECNOLOGÍAS QUE ASUMIREMOS

Tecnología	Para qué sirve
IntelliJ IDEA	Entorno (IDE) donde desarrollamos y ejecutamos el proyecto.
Java	Lenguaje de programación orientado a objetos.
JavaFX	Framework para crear interfaces gráficas.
FXML	Define visualmente la interfaz mediante XML declarativo.
Scene Builder	Diseña las interfaces FXML de forma visual e interactiva.
Maven	Administra las dependencias del proyecto (archivos pom.xml).
MVC	Organiza las responsabilidades (Modelo, Vista, Controlador).
Serializable	Permite guardar objetos en archivos en el disco duro.
DAO	Separa el almacenamiento de los datos de la lógica del programa.

1. ANTES DE COMENZAR: POO APLICADA A INTERFACES

Antes de meternos con JavaFX, tienes que manejar bien algunos conceptos de POO. Una aplicación gráfica sigue siendo un programa orientado a objetos.

Por ejemplo, en un sistema de hotel podríamos tener clases como: Cliente, Reserva, Habitación, Empleado, Producto, Servicio.

```
public class Cliente {
    private String nombre;
    private String correo;

    public Cliente(String nombre, String correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    public String getNombre() { return nombre; }
    public String getCorreo() { return correo; }
}
```

Conceptos en este código

- **Clase:** Cliente
- **Objetos:** Instancias de Cliente
- **Atributos:** nombre, correo
- **Encapsulamiento:** atributos private

¿Por qué importa en interfaces?

Porque una interfaz gráfica normalmente trabaja con objetos. La interfaz no reemplaza la POO; la interfaz **utiliza** la POO para mostrar datos.

2 - 4. IL 2.1 — CONFIGURAR UN PROYECTO PARA INTERFACES IL 2.1

Indicador: Configura un proyecto para interfaces gráficas reconociendo módulos, ciclo de vida y estructura de escenas.

Con **Maven** administramos dependencias en `pom.xml` sin descargar manuales `.jar`:

```
<dependencies>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>...</version>
  </dependency>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>...</version>
  </dependency>
</dependencies>
```

5 - 6. JAVAFX: STAGE, SCENE Y NODE IL 2.1

JavaFX es el framework que nos permite crear interfaces gráficas (Button, Label, TextField, TableView, Alert, etc.). Se organiza formalmente en tres niveles:

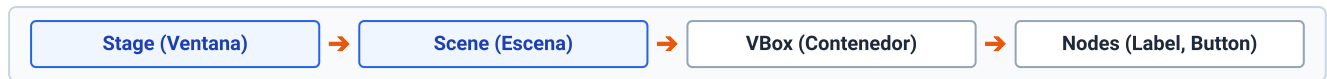
Stage (Ventana)

Es la ventana contenedora principal del sistema operativo.

Scene (Escena)

Es el contenido que se encuentra dentro de la ventana.

Node (Nodos): Son todos los elementos visuales concretos dentro de la escena.



7. EL CICLO DE VIDA DE JAVAFX

Una aplicación JavaFX normalmente parte de una clase que hereda de Application:

```
public class Main extends Application {  
    @Override  
    public void start(Stage stage) {  
        // Construcción e inicio de la interfaz  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Flujo de ejecución:



8. CARGAR UNA INTERFAZ FXML

Normalmente no queremos construir toda la interfaz mediante código Java puro. Por eso utilizamos FXML para cargar la vista:

```
FXMLLoader loader = new FXMLLoader(
    getClass().getResource("/com/ejemplo/principal.fxml")
);

Parent root = loader.load();
Scene scene = new Scene(root);

stage.setScene(scene);
stage.show();
```

Significado: "Carga la interfaz definida en *principal.fxml* y colócala dentro de una *Scene*."

9 - 11. IL 2.2 — DISEÑAR MEDIANTE FXML Y SCENE BUILDER IL 2.2

Indicador: Diseña interfaces gráficas mediante una herramienta visual basada en marcado declarativo, vinculando componentes con el controlador.

FXML: Es un lenguaje basado en XML para describir interfaces de manera *declarativa* (describimos qué contiene, no las instrucciones de ejecución).

```
<VBox>
  <Label text="Nombre"/>
  <TextField/>
  <Button text="Guardar"/>
</VBox>
```

Scene Builder: Permite arrastrar componentes visualmente y genera o modifica automáticamente el archivo FXML correspondiente.

12 - 13. FLUJO INTELLIJ, SCENE BUILDER Y EL CONTROLLER

El flujo de trabajo habitual vincula el diseño visual en FXML con una clase Java denominada Controlador:



En el archivo FXML declaramos qué clase actuará como controlador:

```
<VBox fx:controller="com.ejemplo.controller.PrincipalController">
```

14 - 16. FX:ID, @FXML Y MANEJO DE EVENTOS

Para identificar componentes desde el controlador utilizamos `fx:id`:

```
<!-- En FXML -->
<TextField fx:id="txtNombre"/>
<Button text="Guardar" onAction="#guardar"/>
```

```
// En PrincipalController.java
@FXML
private TextField txtNombre;

@FXML
private void guardar() {
    String nombre = txtNombre.getText();
    System.out.println("Guardado: " + nombre);
}
```

@FXML

Indica que un atributo o método privado será enlazado desde el archivo FXML.

onAction="#guardar"

Vincula el clic del botón con la ejecución del método `guardar()`.

17. IL 2.3 – COMPONENTES, DATOS Y EVENTOS IL 2.3

Uso básico de los principales componentes visuales de JavaFX:

```
// Obtener y modificar texto
lblMensaje.setText("Hola");
String nombre = txtNombre.getText();
String clave = txtClave.getText(); // PasswordField

// Opciones y selecciones
boolean seleccionado = chkActivo.isSelected(); // CheckBox
String ciudad = cmbCiudad.getValue(); // ComboBox
LocalDate fecha = datePicker.getValue(); // DatePicker
```

18. VALIDACIÓN DE DATOS

Una interfaz no debe aceptar datos inconsistentes. Debes aplicar validaciones de campo vacío y captura de excepciones con try-catch:

```
@FXML
private void guardar() {
    if (txtNombre.getText().isBlank()) {
        mostrarError("El nombre es obligatorio.");
        return;
    }

    try {
        int edad = Integer.parseInt(txtEdad.getText());
        if (edad < 18) {
            mostrarError("Debe ser mayor de edad.");
            return;
        }
    } catch (NumberFormatException e) {
        mostrarError("La edad debe ser un número.");
        return;
    }
}
```

19. NOTIFICACIONES CON ALERT

```
Alert alert = new Alert(Alert.AlertType.ERROR);
alert.setTitle("Error");
alert.setHeaderText(null);
alert.setContentText("El nombre es obligatorio.");
alert.showAndWait();
```

20 - 23. IL 2.4 – PATRÓN MVC (MODELO - VISTA - CONTROLADOR) IL 2.4

Indicador: Implementa el patrón MVC para desacoplar responsabilidades en el software.

Modelo (Model)

Representa los datos y conceptos del sistema (clases POO como Cliente).

Vista (View)

Es la interfaz gráfica definida en archivos .FXML.

Controlador (Controller): Recibe los eventos del usuario, valida la entrada y coordina la lógica entre la vista y las capas de datos.

24 - 26. DAO, SERVICE Y ARQUITECTURA COMPLETA

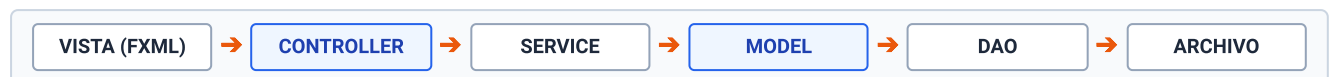
DAO (Data Access Object)

Maneja exclusivamente el almacenamiento (guardar, cargar, eliminar en archivo o BD).

Service

Contiene las reglas de negocio (opcional pero muy recomendable para proyectos robustos).

Arquitectura de capas completa:



27. ESTRUCTURA DEL PROYECTO MVC EN INTELIJ

```
src/main/ ├── java/com/hotel/ | ├── Main.java | ├── model/Cliente.java | ├── controller/ | | ├──
LoginController.java | | ├── ClienteController.java | ├── dao/ClienteDAO.java | ├──
service/ClienteService.java | ├── resources/com/hotel/ ├── login.fxml ├── cliente.fxml
```

28 - 32. IL 2.5 – MOSTRAR COLECCIONES EN TABLEVIEW IL 2.5

Para mostrar colecciones de objetos en un TableView utilizamos listas observables (ObservableList):

```
// En FXML:
<TableView fx:id="tablaClientes">
    <columns>
        <TableColumn fx:id="colNombre" text="Nombre"/>
        <TableColumn fx:id="colCorreo" text="Correo"/>
    </columns>
</TableView>
```

```
// En Controller:
@FXML private TableView<Cliente> tablaClientes;
@FXML private TableColumn<Cliente, String> colNombre;
@FXML private TableColumn<Cliente, String> colCorreo;

@FXML
private void initialize() {
    colNombre.setCellValueFactory(data ->
        new SimpleStringProperty(data.getValue().getNombre()));
    colCorreo.setCellValueFactory(data ->
        new SimpleStringProperty(data.getValue().getCorreo()));
}
```

33 - 34. IL 2.6 – NAVEGACIÓN ENTRE VISTAS IL 2.6

Para cambiar de pantalla instanciamos un nuevo FXMLLoader y reemplazamos la escena actual:

```
FXMLLoader loader = new FXMLLoader(
    getClass().getResource("/com/hotel/clientes.fxml")
);

Parent root = loader.load();

Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
stage.setScene(new Scene(root));
stage.show();
```

35. PASAR INFORMACIÓN ENTRE CONTROLADORES

Para enviar un objeto seleccionado de una pantalla a otra:

```
FXMLLoader loader = new FXMLLoader(
    getClass().getResource("/com/hotel/detalle.fxml")
);
Parent root = loader.load();

// 1. Obtener el controlador de la nueva vista
DetalleController controller = loader.getController();

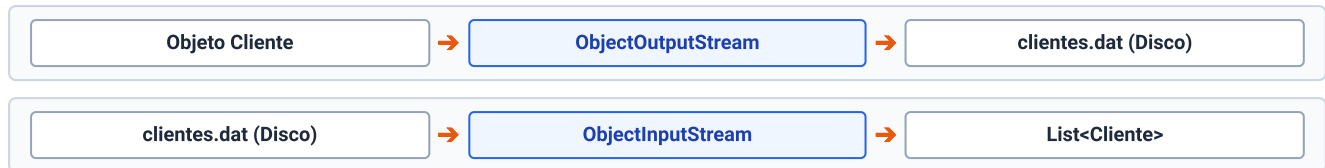
// 2. Transmitir el objeto
controller.setCliente(clienteSeleccionado);

// 3. Mostrar la nueva ventana
Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
stage.setScene(new Scene(root));
stage.show();
```

36 - 40. IL 2.7 – PERSISTENCIA Y SERIALIZACIÓN IL 2.7

Persistencia: Mantiene los datos almacenados en disco aunque el programa se cierre. Para lograrlo mediante archivos de objeto, la clase modelo debe implementar `Serializable`:

```
public class Cliente implements Serializable {
    private static final long serialVersionUID = 1L;
    private String nombre;
    private String correo;
}
```



41 - 42. IMPLEMENTACIÓN DE CLIENTEDAO

```
public class ClienteDAO {
    private final String archivo = "clientes.dat";

    public void guardar(List<Cliente> clientes) throws IOException {
        try (ObjectOutputStream salida = new ObjectOutputStream(
            new FileOutputStream(archivo))) {
            salida.writeObject(clientes);
        }
    }

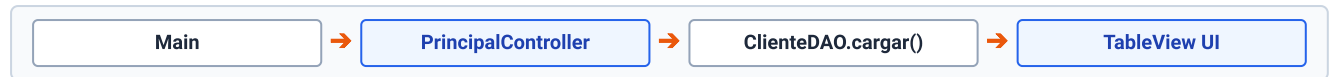
    @SuppressWarnings("unchecked")
    public List<Cliente> cargar() throws IOException, ClassNotFoundException {
        File file = new File(archivo);
        if (!file.exists()) return new ArrayList<>();

        try (ObjectInputStream entrada = new ObjectInputStream(
            new FileInputStream(file))) {
            return (List<Cliente>) entrada.readObject();
        }
    }
}
```

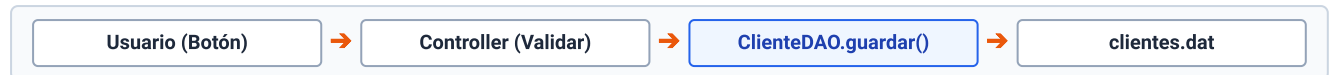
43 - 47. IL 2.8 – PROYECTO COMPLETO E INTEGRACIÓN IL 2.8

Indicador: Desarrolla un componente de software completo aplicando el patrón MVC con persistencia en archivo.

Flujo del sistema completo al iniciar la aplicación:



Flujo al registrar un nuevo elemento:



```
MiProyecto/ ├── pom.xml └── src/main/ ├── java/com/hotel/ | ├── Main.java | ├── model/Cliente.java | ├── controller/PrincipalController.java | ├── service/ClienteService.java | └── dao/ClienteDAO.java └── resources/com/hotel/ └── principal.fxml
```

48. RESUMEN DE CONOCIMIENTOS POR INDICADOR (IL)

IL 2.1 — Entorno y Ciclo IntelliJ, JavaFX, Maven, Stage, Scene, Node, Application, launch(), start().	IL 2.2 — Marcado Declarativo FXML, Scene Builder, fx:controller, fx:id, @FXML, onAction.
IL 2.3 — Componentes y Validación Controls, Eventos, capturar datos, Alert, validación con try-catch.	IL 2.4 — Patrón MVC Diferenciar Model, View, Controller, Service y DAO.
IL 2.5 — Tablas y Colecciones ArrayList, ObservableList, TableView, TableColumn, cellValueFactory.	IL 2.6 — Navegación FXMLLoader, cambiar pantallas, pasar objetos entre controladores.
IL 2.7 — Persistencia DAO Serializable, ObjectOutputStream, ObjectInputStream, DAO.	IL 2.8 — Integración Total Unir entorno, interfaz, MVC, eventos, tablas y persistencia.

49. ERRORES COMUNES EN INTELLIJ Y JAVA FX

Mensaje / Comportamiento	Causa y Solución
"FXML no encontrado"	Revisa <code>getResource()</code> . El FXML debe estar en <code>src/main/resources</code> .
@FXML queda en null	El <code>fx:id</code> en el FXML no coincide con el nombre del atributo en Java.
El botón no responde	Verifica <code>onAction="#guardar"</code> y que el método tenga @FXML.
Controller no encontrado	Revisa que <code>fx:controller="..."</code> tenga el paquete y nombre exacto.
NumberFormatException	Entrada de texto no numérico. Requiere validación previa o bloque try-catch.
Tabla vacía al ejecutar	Faltó hacer <code>tabla.setItems(lista)</code> o configurar <code>cellValueFactory</code> .

50. LA IDEA CLAVE: ENTENDER EL FLUJO DE DATOS

¿Qué ocurre desde que el usuario presiona un botón hasta que los datos terminan guardados en el archivo?



51. ORDEN DE ESTUDIO RECOMENDADO

1. POO → 2. JavaFX / Stage / Scene → 3. FXML / Controller → 4. MVC → 5. TableView → 6. Navegación → 7. DAO / Persistencia.

52. CHECKLIST PARA SABER SI ESTÁS PREPARADO

- ¿Sé explicar la diferencia entre Stage, Scene y Node?
- ¿Sé para qué sirven `fx:id`, `@FXML` y `fx:controller`?
- ¿Sé cómo usar `FXMLLoader` para cambiar de pantalla y transmitir un objeto?
- ¿Por qué el Controller no debe encargarse de escribir el archivo directamente?
- ¿Cómo se conecta un `ObservableList` con un `TableView`?

Resumen final

La interfaz es la puerta de entrada del usuario al programa. El objetivo es construir aplicaciones donde la interfaz, los objetos, los eventos, la lógica y los datos estén correctamente separados y conectados.